

Evaluation Strategies

In pursuit of a probabilistic formulation of linear algebra, it is tempting to start in the same way we started the treatment of integration in Chapter II – by writing down a family of probability measures, say, over H and x , and see if certain choices of parameters can be identified with the classic methods listed above. But due to the structured nature of linear algebra, this approach runs the risk of yielding intractable algorithms. A large part of linear algebra consists of efficient book-keeping. It is thus a better strategy to leave the probabilistic model abstract initially, and focus on algorithmic structure. Concrete implementations informed by the lessons from this section will be addressed in §19.

With this in mind, assume we want to solve the problem $Ax = b$ with a probabilistic solver that mimics the *structure* of an iterative solver like CG (Algorithm 16.1). As noted above, these methods proceed by iteratively collecting *observations* of matrix-vector multiplications $z_i = Ad_i$ of some smartly chosen vector d_i with the matrix A . The vectors d_i are termed *search directions* or *projections*. We will thus consider probabilistic methods that collect action–observation pairs $(d_i, z_i = Ad_i)_{i=0,\dots}$. To simplify notation, we will collect the $d_1, \dots, d_i$ and $z_1, \dots, z_i$ in the columns of two rectangular matrices Z_i, D_i , so that after M iterations of the method, $Z_i, D_i \in \mathbb{R}^{N \times M}$ and the *entire* set of observations can be written compactly as $Z_i = AD_i$. After each iteration, the solver constructs a posterior measure $p(A \mid D_i, Z_i)$, which can be used both to construct an estimate x_i , and to decide on the next projection d_i . For the moment, the only assumption we will make about the as-of-yet unspecified probabilistic inference scheme is that it is consistent with the observation, in the sense that it puts measure zero on all matrices

$\tilde{A}$ for which $\tilde{A}D_i \neq Z_i$. This also implies that any reasonable point estimators A_i and H_i (for $H = A^{-1}$) constructed from this probability measure should have the property

$$A_i D_i = Z_i \quad \text{and} \quad H_i Z_i = D_i.$$

The crucial question is, how should the solver choose the action d_{i+1} from the posterior?

► 17.1 Direct Methods

The most straightforward idea is to choose d_{i+1} a priori, i.e. independent of the observations Z_i . For example along a randomly chosen direction, or as $d_{i+1} = e_{i+1}$, the unit vector $e_{i+1,j} = \delta_{ij}$. One advantage of this strategy is that the directions can be chosen in a way that keeps computational cost low. To wit, sparse directions like the choice $d_i = e_i$ allow the computation of the projections $z_i = Ae_i = A_{:,i}$ in linear time. This idea is at the heart of approximation methods that do not attempt to find an *optimal* solution to a linear problem, but only an ad-hoc, reasonable yet cheap projection. Such approaches can work well if outside information about the structure of the matrix is available. For instance in least-squares problems where the matrix A is constructed by the evaluation of a positive definite function, such as kernel ridge, or Gaussian process, regression. Ideas like the *Nyström* approximation¹ (the name was chosen by Baker,² based on the quadrature work of Nyström),³ *inducing point* methods,⁴ and *spectral* approximations,⁵ and of course Gaussian elimination itself (i.e. LU and Cholesky decompositions), all fall in this category. M individual steps of such methods tend to have cost $\mathcal{O}(NM^2)$, linear in the size of the matrix. The downside of choosing projection directions a priori is that the solver cannot adapt to the structure of the matrix, so if the prior is badly calibrated, the solver's estimate may be bad, too. For the rest of this chapter, we turn our attention to solvers which use collected search directions to converge towards the *exact* solution of the linear problem (defined in Eq. (16.1)). Each iteration of such a method will typically have quadratic cost $\mathcal{O}(N^2)$ because it involves a generic matrix-vector multiplication, so the cost of performing M steps is $\mathcal{O}(N^2M)$.

¹ Williams and Seeger (2000)

² Baker (1973), §3

³ Nyström (1930)

⁴ Snelson and Ghahramani (2005); Quiñero-Candela and Rasmussen (2005).

⁵ Rahimi and Recht (2007)

► 17.2 Iterative Methods

We will take an intuitive approach to constructing an algorithmic skeleton for our adaptive probabilistic solver, which will be

complemented by a subtle improvement later. Our initial developments will not require symmetry of A , but we will change this later. Recall the residual (defined earlier in §16)

$$r(x) = Ax - b.$$

For *any* estimate $\tilde{x}$, wherever it may come from, the update

$$\begin{aligned}\tilde{x} &\leftarrow \tilde{x} - Hr(\tilde{x}) \\ &= \tilde{x} - H(A\tilde{x} - b) = Hb = x\end{aligned}$$

would yield the exact solution, if we had access to the exact matrix inverse H . Since the solver has access to a posterior measure $p(A \mid D_i, Z_i)$, we will assume that this measure can be used to compute some estimate H_i for H . The solver's estimate for the solution x should be consistent with H_i . This suggests the *estimation* update rule

$$x_{i+1} := x_i - H_i r(x_i), \quad (17.1)$$

where the inference on H is so far left abstract.

Following our general recipe, the second part of the solver is the *action* rule, the choice for the next projection d_{i+1} of A . There are two, related but not identical, objectives for this rule: One the one hand, we would like to know the new residual $r(x_{i+1})$, if only to track process and check for convergence (remember that the problem is solved iff $r(x_{i+1}) = 0$). On the other hand, we want to efficiently collect information about A and H ; i.e. explore aspects of A that will maximally improve subsequent estimates $x_{>i}$. To this end, consider the projection and accompanying observation

$$\begin{aligned}d_{i+1} &:= x_{i+1} - x_i = -H_i r(x_i), \\ z_{i+1} &= Ad_{i+1}.\end{aligned} \quad (17.2)$$

An appealing aspect of this choice is that it allows us to compute the new residual without having to evaluate a second matrix-vector multiplication. We can simply update the residual in $\mathcal{O}(N)$ time as

$$r(x_{i+1}) = Ax_{i+1} - b = A(x_{i+1} - x_i + x_i) - b = z_{i+1} + r(x_i).$$

The downside of this action rule is that, without further analysis, there is no guarantee that it will produce particularly informative observations of A . After all, the step $x_{i+1} - x_i$ is just the *greedy* choice that moves the estimate to whatever is currently

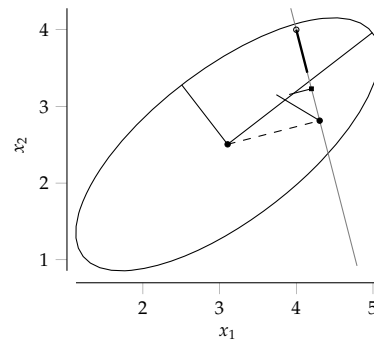


Figure 17.1: A quadratic optimisation problem: extremum as black centre, Hessian with eigen-directions represented by an ellipse with principal axes. The restriction of the quadratic to a linear sub-space is also a quadratic. The optimum in that sub-space can be found in a single division, but it is not identical to the *projection* of the global optimum onto the sub-space.

```

1 procedure LINSOLVE_DRAFT( $A(\cdot)$ ,  $b$ ,  $p(A)$ )
2    $x_0 = H_0 b$  // initial guess
3    $r_0 = Ax_0 - b$  // initial gradient
4   for  $i = 1, \dots, N$  do
5      $d_i = -H_{i-1} r_{i-1}$  // compute optimisation direction
6      $s_i = d_i$  // define projection
7      $y_i = As_i$  // observe
8      $x_i = x_{i-1} + s_i$  // update estimate for  $x$ 
9      $r_i = r_{i-1} + y_i$  // new gradient at  $x_i$ 
10     $H_i = \text{INFER}(H \mid Y_i, S_i, p(A))$  // estimate  $H$ 
11  end for
12 end procedure

```

estimated to be the solution. However, Figure 17.1 illustrates an intuition that might inspire some hope in Eq. (17.2). Recall from (16.3) that the residual is the gradient of the quadratic objective $f(x)$. As such, if we consider the problem from an optimisation perspective, then the residual $r(x_i)$ is the direction of maximal improvement. We will just have to take care that the mapping through the estimate H_i does not destroy this beneficial aspect.

Algorithm 17.1 translates the two design choices of Eqs. (17.1) and (17.2) into more concrete pseudo-code for a first draft of our probabilistic iterative solver.⁶ There is a small change to the lines in Algorithm 17.1 that amounts to a significant improvement in practice. Note that the observation in line 8 of Algorithm 17.1 is available at a point in the loop iteration before x_{i+1} is actually assigned. This offers an opportunity to already use some information from z_i within the loop, as long as it is possible to do so at low computational cost. In particular, we can re-scale the step as $d_{i+1} \leftarrow \alpha_{i+1} d_{i+1}$, using a scalar $\alpha_i \in \mathbb{R}$. Doing so introduces an ever so slight break in the consistency of the probabilistic belief: the estimate x_i in line 8 of Algorithm 17.1 will neither be equal to $H_{i-1}b$ nor to $H_i b$. But this is primarily an issue of algorithmic flow (the fact that x_i and H_i are computed in different lines of the algorithm), and the practical improvements are too big to pass on. In any case, this adaptation is also present in the classic algorithms, so we need to include it to find exact equivalences.

Indeed, under the assumption of symmetric A , the *optimal* scale α_{i+1} can be computed in linear time, using the observation in line 7. We will consider symmetric A hereafter. Consider the parametrised choice $x_i = x_{i-1} + \alpha_i d_i$. The derivative of the

Algorithm 17.1: A draft for a probabilistic iterative linear solver. The notation $A(\cdot)$ is meant to represent a function performing multiplication with A (which does not necessarily require N^2 operations, but might). In this algorithm and others below, the line collecting the observation is marked with a bold-font comment to signify the expensive part of the iteration. Note the structural similarity to CG (Algorithm 16.1).

Line 10 is only a placeholder so far; concrete inference rules will be constructed later. In a practical implementation, lines 5 and 10 would be merged, so that no actual estimate of the quadratic matrix H has to be formed.

⁶ A minor notational complication: line 3 of Algorithm 17.1 wastes one initial matrix-vector multiplication that is not used to infer information about H . This is for generality, so that $x_0 = 0$ (and $H_0 = 0$) remains a possible choice. A related issue one may worry about: Why does the algorithm not simply set $x_i = H_i b$, which may seem like the “natural” estimate for the solution (e.g. it is the mean estimate for x if H is assigned a Gaussian measure)? Note that the update (17.1) can also be written as

$$\begin{aligned}
 x_{i+1} &= x_i - H_i (Ax_i - b) \\
 &= x_i - H_i \left(\sum_{j \leq i} As_j + Ax_0 - b \right) \\
 &= x_0 - H_i Ax_0 + H_i b.
 \end{aligned}$$

Hence, for $x_0 = 0$ (or $H_i Ax_0 = x_0$), Eq. (17.1) is actually equal to $x_i = H_i b$. This is mostly a problem of presentation: Algorithm 17.1 is a compromise allowing both a general probabilistic interpretation while staying close to classic formulations, which typically allow arbitrary x_0 .

```

1 procedure LINSOLVE_PROJECT( $A(\cdot)$ ,  $b$ ,  $p(A)$ )
2    $x_0 = H_0 b$  // initial guess
3    $r_0 = Ax_0 - b$  // initial gradient
4   for  $i = 1, \dots, N$  do
5      $d_i = -H_{i-1} r_{i-1}$  // compute optimisation direction
6      $z_i = Ad_i$  // observe
7      $\alpha_i = -d_i^T r_{i-1} / d_i^T z_i$  // optimal step-size
8      $s_i = \alpha_i d_i$  // re-scale step
9      $y_i = \alpha_i z_i$  // re-scale observation
10     $x_i = x_{i-1} + s_i$  // update estimate for  $x$ 
11     $r_i = r_{i-1} + y_i$  // new gradient at  $x_i$ 
12     $H_i = \text{INFER}(H \mid Y_i, S_i, p(A))$  // estimate  $H$ 
13  end for
14 end procedure

```

Algorithm 17.2: Projected linear optimisation. The variables s, y are redundant with d, z , either pair could be dropped in a real implementation. Both are kept around here to provide uniform notation across algorithms. Note the structural similarity to classic iterative solvers like conjugate gradients (Algorithm 16.1). The only difference is in lines 5 and 12.

convex objective f , (16.2), in this scalar parameter is

$$\begin{aligned} \frac{\partial f(x_{i-1} + \alpha_i d_i)}{\partial \alpha_i} &= \alpha_i d_i^T A d_i + d_i^T (A x_{i-1} - b) \\ &= \alpha_i d_i^T A d_i + d_i^T r_{i-1}. \end{aligned}$$

This derivative is zero at

$$\alpha_i = -\frac{d_i^T A d_i}{d_i^T r_{i-1}}.$$

Recalling that $\nabla f(x) = Ax - b$, note that this α_i defines the point where ∇f is orthogonal to d_i :

$$d_i^T \nabla f(x_{i-1} + \alpha_i d_i) = d_i^T (A x_{i-1} + \alpha_i A d_i - b) = 0.$$

Thus, line 7 of Algorithm 17.1 provides exactly the required matrix-vector multiplication $z_i = Ad_i$ to find the root of the projected gradient. Explicitly,

$$\alpha_i = -\frac{d_i^T r_{i-1}}{d_i^T z_i}. \quad (17.3)$$

Algorithm 17.2 incorporates this insight. It involves only a few re-arrangements from Algorithm 17.1, and one new line of linear cost, to compute the optimal step size α_i . This algorithm will serve as the structural skeleton for the remainder of this chapter. In §19 and following, we will consider several possible choices for the inference step in line 12, and see how they relate to existing classic algorithmic families. In preparation for this step, the following sections give a brief overview over some relevant families of classic linear solvers.

